VIBECODING APLICADO A LA PROGRAMACIÓN CONCURRENTE: GENERACIÓN, ANÁLISIS CRÍTICO Y MÉTRICAS DE UN SISTEMA PRODUCTOR-CONSUMIDOR CON PRIORIDADES

Integrantes:

Máximo JUSTINIANO, Leandro MALLIA, Julián NASPLEDA, Estefania HUERGO, Rodrigo RAMATO

mjustiniano@alumno.unlam.edu.ar ; lmallia@alumno.unlam.edu.ar ; jnaspleda@alumno.unlam.edu.ar ; ehuergo@alumno.unlam.edu.ar ; rramato@alumno.unlam.edu.ar

Asignatura:

(3658) PROGRAMACIÓN CONCURRENTE

Fecha - Versión:

04/10/2026 – Versión 1.0

Resumen:

El presente trabajo aplica la metodología de VibeCoding —delegar la implementación técnica de un programa a un modelo de lenguaje a partir de instrucciones en lenguaje natural— a la generación de un sistema concurrente de complejidad no trivial. Se diseñó un prompt técnico estructurado según el patrón Prompt Pattern (subtipo Input Semantics), especificando lenguaje, plataforma, funcionalidad, requisitos y restricciones para un sistema productor-consumidor con prioridades aplicado a un escenario de procesamiento de pedidos de e-commerce. El código fue generado íntegramente por el modelo Claude Sonnet 5, en Python, usando exclusivamente la biblioteca estándar threading. Se verificó empíricamente su correctitud funcional y de concurrencia, se midieron sus métricas de uso de CPU y memoria bajo dos escenarios de carga, y se evaluó mediante las rúbricas provistas por la cátedra. Los resultados muestran un código funcionalmente correcto desde el primer intento (cero refinamientos), con mecanismos de sincronización adecuados, pero con limitaciones de eficiencia y robustez (selección O(n), ausencia de manejo de excepciones) que requieren revisión humana. Se concluye que el VibeCoding resulta efectivo para programación concurrente cuando el prompt está suficientemente especificado, aunque no reemplaza el análisis crítico experto.

Palabras Clave: *VibeCoding, programación concurrente, productor-consumidor, prompt técnico, métricas de software.*

I. CONTEXTO

El presente trabajo se enmarca en la cátedra de Programación Concurrente de la carrera de Ingeniería en Informática de la Universidad Nacional de La Matanza (UNLaM), como parte de la Actividad Práctica Obligatoria N.° 1 del segundo cuatrimestre de 2026. Se inscribe, además, en una línea de investigación más amplia de la cátedra sobre la aplicación de herramientas de generación automática de código mediante Inteligencia Artificial Generativa (VibeCoding) en distintos escenarios de desarrollo de software, de la cual forma parte el trabajo preliminar de Carnuccio et al. [6], que analiza diversas herramientas de VibeCoding aplicadas a desarrollo móvil, sistemas embebidos y programación concurrente en Python.

II. INTRODUCCIÓN

La programación concurrente estudia la ejecución simultánea o intercalada de múltiples flujos de control que comparten recursos, y ha sido un área central de la informática desde los trabajos fundacionales de Dijkstra sobre procesos cooperantes y el uso de semáforos para resolver el problema del productor-consumidor [1]. Posteriormente, Hoare formalizó el concepto de monitor como una estructura de más alto nivel para encapsular el estado compartido y su sincronización [2], antecedente directo de las primitivas Lock y Condition que ofrece hoy la biblioteca threading de Python.

En paralelo, en los últimos años la Inteligencia Artificial Generativa irrumpió como herramienta de asistencia a la programación. Karpathy acuñó en febrero de 2025 el término “vibe coding” para describir un nuevo paradigma de desarrollo en el que el programador describe en lenguaje natural lo que necesita y delega en un modelo de lenguaje la implementación técnica [7]. Si bien este enfoque promete democratizar el acceso a la programación, diversos estudios empíricos advierten sobre riesgos de calidad: Pearce et al. encontraron que una proporción significativa del código sugerido por GitHub Copilot contenía vulnerabilidades de seguridad clasificables según el catálogo CWE [3], mientras que Yetiştiren et al. evaluaron la corrección, eficiencia y seguridad del código generado por Copilot y hallaron que, si bien gran parte del código es funcionalmente correcto, persisten deficiencias de calidad que exigen revisión humana [4]. A nivel cognitivo, Kosmyna et al. reportaron una menor conectividad neuronal y una “deuda cognitiva” en quienes recurren sistemáticamente a asistentes de IA para tareas de escritura, lo que motiva preguntas análogas sobre su uso en programación [5].

En este contexto, el presente trabajo busca aportar evidencia empírica específica sobre la aplicación de VibeCoding a un dominio particularmente exigente: la programación concurrente, donde los errores de sincronización suelen ser no determinísticos y difíciles de detectar mediante pruebas superficiales.

III. MÉTODOS

Se siguió una metodología de tres etapas. En primer lugar, se diseñó un prompt técnico estructurado según el patrón Prompt Pattern, en su variante Input Semantics (Meta Language Creation), organizado en las cinco secciones fijas propuestas por la cátedra: Lenguaje de Programación, Plataforma, Funcionalidad a Implementar, Requisitos Específicos y Restricciones [6]. El problema elegido fue una variante no trivial del clásico productor-consumidor [1]: un sistema de procesamiento de pedidos de e-commerce con dos niveles de prioridad (urgente/normal) y un mecanismo de envejecimiento (aging) para evitar inanición, implementado exclusivamente con primitivas de threading (Lock, Condition) sin ningún framework de terceros.

En segundo lugar, el prompt se envió al modelo Claude Sonnet 5 (Anthropic), a través del entorno Cowork, que generó el código completo en una única iteración. El código resultante se verificó empíricamente compilándolo y ejecutándolo repetidas veces, y se diseñó además una prueba aislada y determinística sobre la función de selección de pedidos (elegir_mejor_pedido) para verificar el mecanismo de envejecimiento sin depender del azar de los tiempos de ejecución.

En tercer lugar, se aplicaron las métricas solicitadas por la cátedra. El cumplimiento funcional y la correctitud de los mecanismos de concurrencia (sincronización, ausencia de deadlocks, ausencia de condiciones de carrera, exclusión mutua) se evaluaron con las rúbricas provistas (escala Alta/Media/Baja, equivalente a los niveles A/B/C de las rúbricas). El uso de CPU y memoria se midió con la biblioteca psutil, monitoreando el proceso del sistema operativo desde afuera (sin instrumentar el código medido) con muestreo cada 100 ms, en dos escenarios: carga normal (4 productores, 3 consumidores, 50 pedidos) y carga alta (8 productores, 6 consumidores, 300 pedidos). Finalmente, el consumo de tokens se estimó matemáticamente a partir de la cantidad de caracteres del prompt técnico y del código generado, dividida por un promedio de 4 caracteres por token, heurística estándar utilizada por los proveedores de modelos de lenguaje.

Además de la verificación empírica mediante ejecuciones repetidas, se aplicó un argumento estructural para la ausencia de deadlocks: el programa nunca mantiene dos locks tomados simultáneamente (el Lock del buffer, el de ContadorAtomico y el de RecolectorDeMetricas se usan de forma completamente independiente, sin anidarse nunca), lo que elimina por construcción la espera circular, una de las cuatro condiciones de Coffman necesarias para que exista un deadlock. Es, por lo tanto, una demostración y no una simple observación empírica.

Por último, para evaluar la robustez del código ante fallos —una dimensión que las rúbricas provistas no contemplan explícitamente— se diseñó una prueba de inyección de fallas: se sustituyó temporalmente el método de registro de métricas para que lanzara una excepción no capturada la primera vez que se invocara, simulando un error inesperado durante el procesamiento de un pedido, y se observó el comportamiento resultante del sistema completo.

IV. RESULTADOS Y OBJETIVOS

El código generado cumplió la totalidad de los nueve requisitos específicos y las cinco restricciones del prompt técnico, verificado empíricamente y no solo por inspección: compiló sin errores, se ejecutó correctamente en corridas independientes, respetó la capacidad acotada del buffer, priorizó los pedidos urgentes sin dejar en inanición a los normales, y finalizó de forma ordenada mediante el patrón poison pill en 8 a 9 segundos, muy por debajo del límite de 60 segundos impuesto por la consigna. Aplicando el esquema de puntaje de cada rúbrica (ver Métodos), el proyecto obtuvo 90/100 en la rúbrica de funcionalidad y 90/100 en la rúbrica de concurrencia. La Tabla 1 resume estos resultados junto con las métricas de recursos.

Métrica	**Resultado**
Funciona sin errores (rúbrica funcionalidad)	A — Suficientemente logrado (40/40 pts)
Cumplimiento del objetivo funcional (rúbrica funcionalidad)	A — Suficientemente logrado (40/40 pts)
Diseño de la aplicación (rúbrica funcionalidad)	B — Medianamente logrado (10/20 pts)
Sincronización correcta (rúbrica concurrencia)	A — Suficientemente logrado (25/25 pts)
Ausencia de deadlocks (rúbrica concurrencia)	A — Suficientemente logrado (25/25 pts)
Ausencia de condición de carrera (rúbrica concurrencia)	A — Suficientemente logrado (25/25 pts)
Uso correcto de exclusión mutua (rúbrica concurrencia)	B — Medianamente logrado (15/25 pts)
CPU promedio — carga normal	0,51 %
CPU promedio — carga alta	0,87 %
Memoria promedio — carga normal	13,55 MB
Memoria promedio — carga alta	13,70 MB
Refinamientos necesarios sobre el prompt	0
Tokens estimados (prompt + código)	≈ 6.434
Puntaje total — rúbrica de funcionalidad (sobre 100)	**90/100**
Puntaje total — rúbrica de concurrencia (sobre 100)	**90/100**

TABLA 1.

RESULTADOS DE RÚBRICAS Y MÉTRICAS DE RECURSOS

El uso de CPU se mantuvo por debajo del 1 % en ambos escenarios (0,51 % en carga normal, 0,87 % en carga alta), lo que confirma que la carga simulada (mediante time.sleep()) es de naturaleza I/O-bound y no se ve limitada por el Global Interpreter Lock (GIL) de CPython, a diferencia de lo que ocurriría con una carga de cómputo puro. En cuanto a la memoria, el incremento fue marginal (de 13,55 MB a 13,70 MB) pese a multiplicar por seis la cantidad de pedidos procesados (de 50 a 300), lo que evidencia que el buffer acotado cumple su función de limitar el crecimiento de memoria independientemente del volumen total procesado.

El análisis crítico del código identificó, no obstante, una limitación que motivó bajar de 100 a 90 puntos la rúbrica de concurrencia: la función _elegir_mejor_pedido() recorre linealmente todo el buffer (complejidad O(n)) y list.remove() realiza otro recorrido lineal, ambos dentro de la sección crítica protegida por el lock. La exclusión mutua protege así más cómputo del estrictamente necesario, lo que —según la propia rúbrica de concurrencia— corresponde a una protección “parcial o excesiva” (nivel B, 15/25 pts) y no a una protección correcta (nivel A). Adicionalmente, el uso de notify_all() en lugar de notify() despierta innecesariamente a todos los hilos en espera, ineficiencia conocida en la literatura como “thundering herd”; esta última no afecta ningún criterio puntuado por la rúbrica, pero queda documentada como limitación de diseño.

El hallazgo más relevante del análisis crítico, sin embargo, es de robustez y no de concurrencia: la prueba de inyección de fallas descripta en Métodos confirmó empíricamente que el código carece de manejo de excepciones. Al forzar una excepción durante el procesamiento de un pedido, el hilo consumidor afectado terminó de forma abrupta y silenciosa —sin que join() lo reportara como error— y el pedido en curso se perdió sin dejar rastro en el resumen final: de 20 pedidos generados, solo 19 fueron contabilizados como procesados. Esta limitación no está contemplada por ninguna de las dos rúbricas aplicadas (que evalúan sincronización y funcionalidad bajo condiciones normales, no tolerancia a fallos), por lo que no resta puntaje en la Tabla 1, pero constituye, a criterio de los autores, el hallazgo más importante del análisis crítico: el código generado es correcto y eficiente bajo condiciones normales, pero no es robusto frente a errores no previstos, confirmando empíricamente las advertencias de la literatura sobre la necesidad de revisión humana del código generado por IA [3], [4].

Como objetivo en curso, se plantea reproducir el experimento con otras herramientas de VibeCoding (ChatGPT, GitHub Copilot, Gemini) para comparar la tasa de refinamientos necesarios y la calidad del código resultante ante el mismo prompt técnico, en línea con el enfoque comparativo iniciado por la cátedra [6].

V. CONCLUSIONES

El VibeCoding demostró ser una herramienta efectiva para generar un programa concurrente de complejidad no trivial, funcionalmente correcto y con mecanismos de sincronización adecuados, sin necesidad de refinamientos adicionales sobre el prompt original. Esto sugiere que la calidad del código generado depende fuertemente de la calidad y el nivel de especificidad del prompt: un prompt técnico estructurado, que fija explícitamente el lenguaje, la plataforma, los requisitos y las restricciones, reduce la ambigüedad que —según reportan Yetiştiren et al. [4] y el propio relevamiento de la cátedra [6]— suele forzar múltiples iteraciones de corrección con otras herramientas. Sin embargo, la ausencia de errores funcionales no implica ausencia de limitaciones: el análisis crítico detectó decisiones de diseño subóptimas en términos de eficiencia (selección lineal, notificación amplia) y robustez (sin manejo de excepciones, confirmada empíricamente mediante una prueba de inyección de fallas, ver Métodos y Resultados) que un desarrollador humano con experiencia en sistemas concurrentes probablemente habría señalado o evitado desde el diseño. Esto confirma, en un dominio de alta exigencia técnica como la programación concurrente, el rol del desarrollador como “validador” del código producido por la IA, en lugar de su creador directo, el cambio de rol que ya señalan Pearce et al. [3] y Karpathy [7] respecto del paradigma de VibeCoding. En definitiva, la IA generativa no reemplaza el criterio experto en programación concurrente, pero sí puede acelerar significativamente la primera versión funcional de un sistema, liberando tiempo del desarrollador para las tareas de revisión crítica, optimización y endurecimiento ante fallos que el propio análisis de este trabajo puso de manifiesto.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas:

[1] E. W. Dijkstra, "Cooperating Sequential Processes," Technological University Eindhoven, Rep. EWD-123, 1965. Reimpreso en F. Genuys (Ed.), Programming Languages, Academic Press, 1968, pp. 43-112.

[2] B. Yetiştiren, I. Özsoy, E. Tüzün, "Assessing the Quality of GitHub Copilot's Code Generation," Proc. 18th Int. Conf. on Predictive Models and Data Analytics in Software Engineering (PROMISE '22), ACM, pp. 62-71, nov. 2022.

[3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," Proc. 2022 IEEE Symposium on Security and Privacy (SP), IEEE, pp. 754-768, may. 2022.

[4] C. A. R. Hoare, "Monitors: An Operating System Structuring Concept," Communications of the ACM, vol. 17, no. 10, pp. 549-557, oct. 1974.

[5] N. Kosmyna, E. Hauptmann, Y. T. Yuan, J. Situ, X.-H. Liao, A. V. Beresnitzky, I. Braunstein, P. Maes, "Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task,", jun. 2025.

[6] E. Carnuccio et al., "Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software," Departamento de Ingeniería e Investigaciones Tecnológicas, Universidad Nacional de La Matanza (UNLaM), San Justo, Argentina, [2025/2026].

[7] A. Karpathy, "There's a new kind of coding I call 'vibe coding'...", publicación en X (anteriormente Twitter), 2 feb. 2025. [En línea]. Disponible en: https://x.com/karpathy/status/1886192184808149383

B. Bibliografía:

No aplica: la totalidad de las fuentes consultadas para este trabajo se encuentra citada en el apartado A.